Threads in Java: what to write, what never to write, and why

Programming Paradigms, session 7. Ten rules. Each has a program that breaks it and one that keeps it, in `Rules.java`, and a line from a run. The run shows the failure; the argument says why. We run them ourselves: `java Rules <rule>`, or `all`.

---

1. A lock protects data only if every access takes it every-access

A lock is a rule we follow: every read and every write of the data it protects happens while we hold it. One access outside the rule is enough; the data is then shared with no protection at all, however careful the other accesses are.

```
class Counter {
  int n;
  synchronized void inc() { n++; }
  void add(int k) { n += k; }      // forgot the lock
  synchronized int get() { return n; }
}
```

Never protect some of the accesses to a piece of data.

Write one lock per piece of data, named beside it, and take it in every method that touches the data.

Who checks nobody. Rust ties the data to its lock, so that there is no way to reach it without holding it.

One run: broken 754915 of 1000000; fixed 1000000 of 1000000

---

2. Two locked steps are not one locked operation two-steps

A transfer takes from one account and gives to another. If each step takes its own lock and releases it, there is no data race anywhere, and the program is still wrong: between the two steps the money is nowhere, and any thread that looks sees it. The invariant, the total is constant, spans both accounts, so the operation must hold both locks for its whole extent. This is an atomicity violation, and no lock discipline on single accesses removes it.

```
// broken: each half under its own lock
synchronized (from) { from.balance -= k; }
synchronized (to)   { to.balance   += k; }

// fixed: both locks, for the whole operation
synchronized (a) { synchronized (b) {
  from.balance -= k; to.balance += k;
} }
```

Never release a lock in the middle of an operation whose invariant spans the data it protects.

Write the invariant first; then choose the critical section so that the invariant is true at its start and at its end, and hold every lock the invariant needs for the whole of it.

Who checks nobody. Not Rust either: this program compiles in safe Rust and is wrong in the same way. A data race is what a compiler can rule out; an atomicity violation is ours.

One run: broken audits that saw a wrong total: 237619 of 250000; fixed audits that saw a wrong total: 0 of 250000

---

3. The plain cell is not atomic volatile

An increment is a read, an addition and a write. The atomic cell offers the three as one instruction; the plain cell offers nothing between threads. In Java, `volatile` makes each read and each write visible to other threads in order, and does nothing for the three together. `AtomicInteger` offers the whole read-modify-write as one instruction.

```
static volatile int n;
n++;                              // three steps, still a race

AtomicInteger n = new AtomicInteger();
n.incrementAndGet();              // one instruction
```

Never write `n++` on a shared field, volatile or not.

Write an atomic cell for a counter or a flag; a lock when more than one word must change together.

Who checks nobody. The cost is bounded: a wrong number, never a corrupted heap. In C and C++ the same program has no defined meaning.

One run: broken 265937 of 1000000; fixed 1000000 of 1000000

---

4. Wait in a loop, never in an `if` wait-while

A thread that waits for a condition gives up the lock while it waits and takes it back before it goes on. Between the wake-up and the lock, another thread may have taken the lock and made the condition false again. A wake-up with no cause is allowed too. So the condition is tested again after every wake-up, which a loop does and an `if` does not.

```
synchronized void put() throws InterruptedException {
  while (count == capacity) wait();   // never: if (...) wait();
  count++;
```

```
  notifyAll();
}
```

Never `if (cond) wait();`

Write `while (!ready) wait();`

Who checks nobody. The bounded buffer of capacity one, two producers and two consumers, overflows under the `if`.

One run: broken stuck at the deadline; overflows 13937, underflows 13940; fixed finished; overflows 0, underflows 0

---

5. Wake everyone, unless we can argue for one notify-all

A single wake-up rouses one waiting thread, chosen by the runtime. If the one woken cannot use the news, a producer woken by a producer, the news is lost, and every thread may end up waiting. Waking all costs a few useless wake-ups; each thread re-tests its condition by rule 4 and goes back to waiting if the news was not for it.

```
notify();      // wakes one; a producer may wake a producer
notifyAll();   // wakes all; each re-tests its condition
```

Never wake one thread when more than one kind of thread waits on the same condition.

Write `notifyAll` by default; one condition per kind of waiter and `notify` only with the argument written down.

Who checks nobody. The same buffer, with the loop but `notify`, stops.

One run: broken stuck at the deadline; overflows 0, underflows 0; fixed finished; overflows 0, underflows 0

---

6. Take several locks in one order, everywhere lock-order

Two threads, two locks, each holding one and waiting for the other: neither can move. Mutual exclusion still holds, nothing bad has happened, and nothing good will ever happen again. Deadlock is a failure of liveness. The classic cure is a total order on the locks, kept by every thread.

```
// thread 1              // thread 2
synchronized (a) {       synchronized (b) {
  synchronized (b) {}      synchronized (a) {}
}                        }
// fixed: both take a, then b
```

Never take lock B while holding A in one place and A while holding B in another.

Write the order down, by an identifier or an address, and a helper that takes two locks in that order.

Who checks nobody, and no compiler on the table: Rust compiles this deadlock too. A detector can find it after the fact.

One run: broken stuck at the deadline: deadlock; fixed finished

---

7. Compare-and-set compares what we read, not a value we rebuilt cas-identity

A compare-and-set succeeds when the cell still holds the very object we expect. A value rebuilt from the same number is another object, and the compare-and-set fails although the numbers agree. Every correct retry loop compares against the value it read. Java compares by identity: a fresh `Integer` is not the one inside the reference.

```
AtomicReference<Integer> r = new AtomicReference<>(1000);
r.compareAndSet(1000, 1001);     // false: a fresh Integer
Integer v = r.get();
r.compareAndSet(v, v + 1);       // true
```

Never pass a literal or a freshly built value as the expected argument of a compare-and-set.

Write `Integer v = r.get(); r.compareAndSet(v, f(v))`, and loop on failure. `AtomicInteger` compares by value, because the number is the object.

Who checks nobody; the type checker accepts both calls.

One run: broken compareAndSet(1000, 1001) on a reference holding 1000: false; fixed compareAndSet(v, v + 1) with v the value read: true

---

8. A right answer in a run proves nothing runs

The racy counter with a thousand increments per thread prints the right number in almost every run; the same program with more increments is wrong in most. The program did not change; the schedule did. Correct means: in every execution. A run can show a bug; only an argument shows there is none.

```
int[] cell = new int[1];
// four threads: for (…) cell[0]++;
// 1000 each: right in most runs; 250000 each: wrong in most
```

Never conclude from a run, or from a hundred, that a shared structure is right.

Write the invariant and the point where each operation takes effect; then test with many threads, many operations, many runs, to catch what the argument missed.

Who checks nobody here. Go's race detector reports a race it saw in a run; Rust refuses the data race before any run; for the atomicity violations of rule 2, nobody.

One run: small 1000 increments per thread: wrong in 1 of 100 runs; large 250000 increments per thread, same program: wrong in 5 of 10 runs

---

9. A write another thread must see goes through a volatile or an atomic visibility

A thread that spins on a plain cell written by another thread is a race, and what a race gives is the language's contract to state, not ours to guess. The Java compiler is allowed to read a plain field once and keep it in a register, and does: the loop never ends. `volatile` forbids that.

```
static boolean stop;              // broken: may spin forever
static volatile boolean stop;     // fixed
new Thread(() -> { while (!stop) { } }).start();
```

Never pass a flag or a result between threads through a plain field.

Write `volatile` or an atomic for a flag; a lock, a queue or a join for anything larger. A value published before a volatile write is visible after a volatile read of the same field.

Who checks nobody. The compiler is what makes the broken loop spin forever: read the language's contract before relying on a plain field.

One run: broken the loop never saw the write: stuck at the deadline; fixed the loop ended

---

10. A Java thread is cheap many-threads

The unit that runs in parallel is not the same thing in every language, and the number we may create follows from what it is. Java starts ten thousand platform threads and joins them; a virtual thread is cheaper still.

```
for (int i = 0; i < 10_000; i++) new Thread(task).start();   // fine
Thread.startVirtualThread(task);                             // cheaper still
```

Never assume a thread is free: ten thousand platform threads hold ten thousand stacks.

Write a pool, or virtual threads, for many small tasks.

Who checks nobody; the machine answers with memory.

One run: fixed 10000 threads started and joined: true